

Spark Optimization Details

By Dhandapani Yedappalli Krishnamurthi Sep 10, 2025

Code-Level Optimizations

1. Avoid Using `collect()` Unnecessarily

Explanation:

`collect()` pulls the entire dataset into the **driver memory**. This can cause **OutOfMemory (OOM)** if the dataset is large.

Pros:

- Useful for debugging small datasets.
- Gives full control of data on the driver.

Cons:

- Dangerous for big data → crashes drivers.
- Breaks the distributed nature of Spark.

When to Use:

- When the dataset is **tiny (<100 MB)** and you really need it on the driver (e.g., export to Pandas for ML).

When NOT to Use:

- On production datasets (GBs or TBs). Use `show()`, `take()`, or write to storage instead.
-

2. Use `mapPartitions` Instead of `map`

Explanation:

`map()` runs per record; `mapPartitions()` runs per partition.

Pros:

- Reduces expensive setup (e.g., DB connections).
- Faster execution since initialization happens once per partition.

Cons:

- Harder to write than `map`.
- Risk: if one partition is too large, it can cause memory issues.

When to Use:

- Writing to **external systems (DB, API calls)**.
- Expensive object creation (e.g., ML model loading).

When NOT to Use:

- If data per partition is huge → can overload memory.

3. Broadcast Joins

Explanation:

Broadcasts a **small lookup table** to all executors to avoid shuffle joins.

Pros:

- Extremely fast join (no shuffle).
- Good for star schema (fact table + dimension).

Cons:

- Broadcast table stored in memory → OOM if too large.
- Not good for frequently updated lookup tables.

When to Use:

- Lookup tables < **100MB**.
- Dimension table joins.

When NOT to Use:

- If both tables are large (Spark shuffle is safer).

4. Cache/Persist Reused Data

Explanation:

Store DataFrame/RDD in memory/disk for reuse.

Pros:

- Avoids recomputation.
- Faster for iterative algorithms (ML, graph processing).

Cons:

- Uses executor memory (may cause eviction & GC).
- Wrong usage → memory pressure.

When to Use:

- When DataFrame is reused multiple times in the same job.

When NOT to Use:

- If the dataset is used only once.
-

5. Avoid UDFs, Use Spark SQL Functions

Explanation:

Built-in functions are optimized by **Catalyst**; UDFs are black-box.

Pros:

- Catalyst can optimize built-in functions.
- Better performance (vectorized operations).

Cons:

- UDFs offer flexibility.
- Hard to avoid if there is no built-in equivalent.

When to Use:

- Always prefer Spark SQL functions (`upper`, `substring`, `date_format`).

When NOT to Use:

- Only when business logic is too complex.
-

6. Partition Data Before Writing

Explanation:

Partitioning creates multiple files based on column values.

Pros:

- Faster reads with **partition pruning**.
- Better parallelism when writing.

Cons:

- Too many partitions = small files problem.
- Over-partitioning wastes storage.

When to Use:

- On partition columns (e.g., **year**, **month**, **region**).

When NOT to Use:

- High-cardinality columns (e.g., **customer_id**).
-

7. Optimize Shuffle Partitions

Explanation:

`spark.sql.shuffle.partitions` controls number of output partitions in shuffle operations (joins, groupBy).

Pros:

- Prevents too many small tasks.
- Improves resource utilization.

Cons:

- Too few partitions = stragglers.
- Too many partitions = overhead.

When to Use:

- Tune based on dataset size (rule of thumb: partition size ~128MB).

When NOT to Use:

- Leaving it at default (200) is often inefficient.

8. Push Down Filters & Column Pruning

Explanation:

Apply filters early & select only needed columns.

Pros:

- Reduces data read & shuffle.
- Faster queries.

Cons:

- None, except risk of filtering incorrectly.

When to Use:

- Always (best practice).

When NOT to Use:

- Rarely, unless you need all columns for further transformations.
-

9. Repartition vs Coalesce

Explanation:

- `repartition(n)` → full shuffle (expensive).
- `coalesce(n)` → reduces partitions (no full shuffle).

Pros:

- Repartition → better parallelism for writers.
- Coalesce → efficient for reducing files.

Cons:

- Repartition is expensive.
- Coalesce may cause skew if reduced too much.

When to Use:

- Repartition before shuffle-heavy operations.
- Coalesce before writing a small dataset.

When NOT to Use:

- Repartition unnecessarily (wastes time).
-

Cluster-Level Optimizations

1. Executor & Core Tuning

Explanation:

Balance memory & CPU cores across executors.

Pros:

- Efficient resource usage.
- Avoids long GC pauses.

Cons:

- Wrong config = wasted resources.

When to Use:

- Benchmark jobs with different configs (trial-and-error).

When NOT to Use:

- Leaving everything to defaults (bad scaling).
-

2. Dynamic Allocation

Explanation:

Spark adjusts executors dynamically.

Pros:

- Saves cluster resources.
- Useful in multi-tenant clusters.

Cons:

- May cause executor churn (frequent adds/removes).
- Requires shuffle service enabled.

When to Use:

- In shared clusters.

When NOT to Use:

- Latency-sensitive jobs (executor churn = delays).

3. Choose Right File Format

Explanation:

Parquet/ORC > CSV/JSON.

Pros:

- Columnar storage = faster reads.
- Compression = smaller size.

Cons:

- CSV better for raw text sharing.

- JSON better for semi-structured data.

When to Use:

- Analytical workloads → Parquet + Snappy.

When NOT to Use:

- If consumers need raw CSV/JSON.
-

4. Data Locality

Explanation:

Run tasks close to data.

Pros:

- Reduces network transfer.
- Faster job execution.

Cons:

- Sometimes cluster manager ignores locality for load balancing.

When to Use:

- HDFS/YARN environments.

When NOT to Use:

- Cloud storage (S3, GCS) → locality doesn't apply.
-

5. Adaptive Query Execution (AQE)

Explanation:

Spark 3.x runtime optimization (auto join selection, partition merging).

Pros:

- Handles skew automatically.
- Faster queries.

Cons:

- Overhead of runtime decisions.
- Only available Spark 3+.

When to Use:

- Joins, aggregations, skewed data.

When NOT to Use:

- Very small jobs (overhead > benefit).

6. Avoid Small Files Problem

Explanation:

Lots of small files slow down HDFS/S3 reads.

Pros:

- Improves scan performance.
- Better parallelism.

Cons:

- Coalescing may reduce parallelism too much.

When to Use:

- After heavy partitioned writes.

When NOT to Use:

- Streaming jobs (naturally create small files).
-

7. Serialization (Kryo)

Explanation:

Kryo is faster & smaller than Java serialization.

Pros:

- Less memory & disk usage.
- Faster shuffle.

Cons:

- Must register custom classes.
- Not default.

When to Use:

- Big RDD shuffles.

When NOT to Use:

- Small jobs (no noticeable gain).
-

8. Shuffle & Spill Optimization

Explanation:

Reduce shuffle writes & disk spill.

Pros:

- Faster queries with large joins/groupBy.

Cons:

- Needs careful memory tuning.

When to Use:

- Heavy aggregation workloads.

When NOT to Use:

- Small datasets.
-